School of Computer Science and Artificial Intelligence

Lab Assignment-9.1

Course Name : AI Assistant Coding
Name of the Student : MOHAMMAD ADIL
Registration No : 2303A54042
Batch No : 48

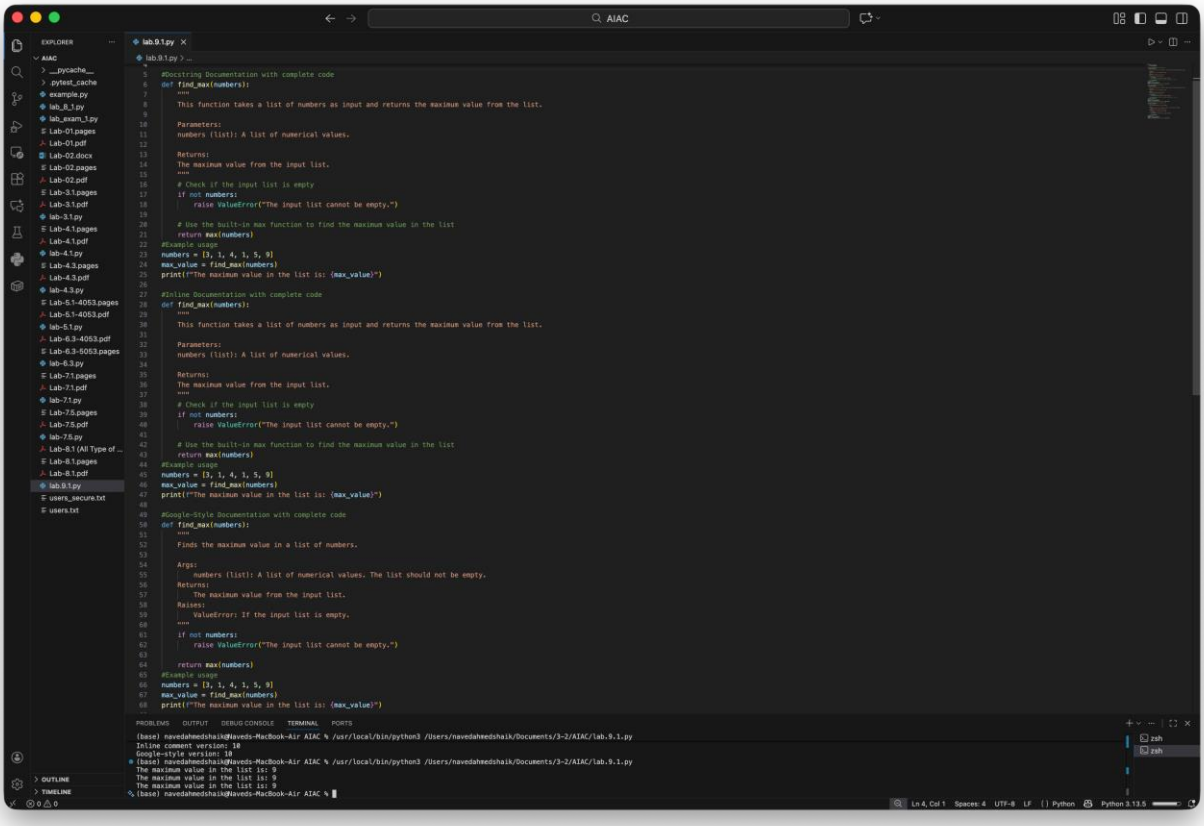
Problem 1:Consider the following Python function: def find_max(numbers): return max(numbers) Task:

- Write documentation for the function in all three formats:

(a) Docstring
(b) Inline comments
(c) Google-style documentation

- Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
- Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

SCREENSHOT OF GENERATED CODE:



```
#Docstring Documentation with complete code
def find_max(numbers):
    """
    This function takes a list of numbers as input and returns the maximum value from the list.

    Parameters:
    numbers (list): A list of numerical values.

    Returns:
    The maximum value from the input list.
    """
    # Check if the input list is empty
    if not numbers:
        raise ValueError("The input list cannot be empty.")

    # Use the built-in max function to find the maximum value in the list
    return max(numbers)
#Example usage
numbers = [3, 1, 4, 1, 5, 9]
max_value = find_max(numbers)
print(f"The maximum value in the list is: {max_value}")

#Inline Documentation with complete code
def find_max(numbers):
    """
    This function takes a list of numbers as input and returns the maximum value from the list.

    Parameters:
    numbers (list): A list of numerical values.

    Returns:
    The maximum value from the input list.
    """
    # Check if the input list is empty
    if not numbers:
        raise ValueError("The input list cannot be empty.")

    # Use the built-in max function to find the maximum value in the list
    return max(numbers)
#Example usage
numbers = [3, 1, 4, 1, 5, 9]
max_value = find_max(numbers)
print(f"The maximum value in the list is: {max_value}")

#Google-Style Documentation with complete code
def find_max(numbers):
    """
    Finds the maximum value in a list of numbers.

    Args:
        numbers (list): A list of numerical values. The list should not be empty.
    Returns:
        The maximum value from the input list.
    Raises:
        ValueError: If the input list is empty.
    """
    if not numbers:
        raise ValueError("The input list cannot be empty.")

    return max(numbers)
#Example usage
numbers = [3, 1, 4, 1, 5, 9]
max_value = find_max(numbers)
print(f"The maximum value in the list is: {max_value}")
```

```
(base) navedahmedshaik@Naveds-MacBook-Air AIAC % /usr/local/bin/python3 /Users/navedahmedshaik/Documents/3-2/AIAC/lab.9.1.py
Inline comment version: 10
Google-style version: 10
(base) navedahmedshaik@Naveds-MacBook-Air AIAC % /usr/local/bin/python3 /Users/navedahmedshaik/Documents/3-2/AIAC/lab.9.1.py
The maximum value in the list is: 9
The maximum value in the list is: 9
The maximum value in the list is: 9
(base) navedahmedshaik@Naveds-MacBook-Air AIAC %
```

EXPLANATION:

The function `find_max` takes a list of numbers as input and returns the maximum value from that list. It first checks if the input list is empty and raises a ValueError if it is. If the list is not empty, it uses the built-in `max` function to find and return the maximum value. The function is documented using three different styles: docstring documentation, inline documentation, and Google-style documentation, providing clear explanations of the function's purpose, parameters, return value, and potential exceptions.

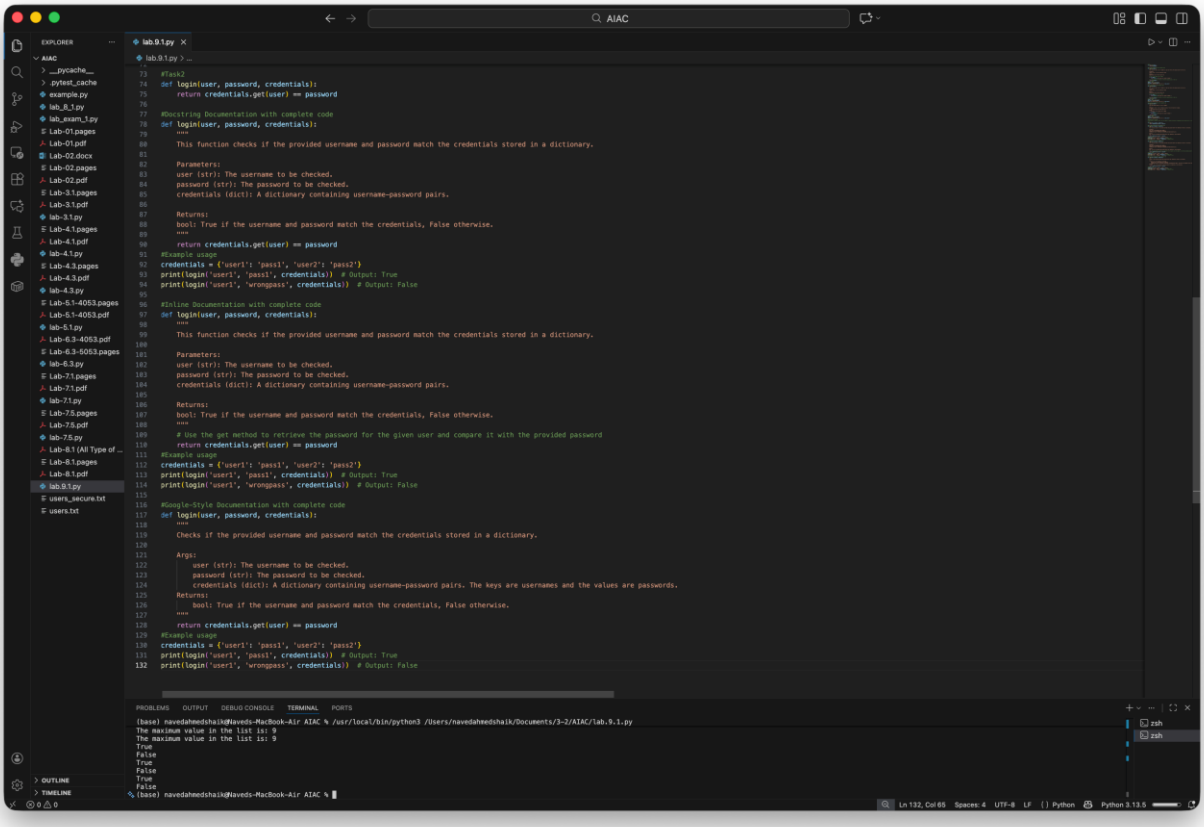
Style	Advantages	Disadvantages	Suitable Use Cases
Docstring (standard)	Simple, readable, built into Python help tools	Less structured than formal styles	Small scripts, educational code
Inline comments	Explain logic step-by-step	Can clutter code, not accessible via help()	Complex algorithms needing explanation
Googlestyle	Highly structured, professional, IDE-friendly, good for teams	Slightly longer to write	Libraries, APIs, production code

Problem 2: Consider the following Python function:

def login(user, password, credentials): return credentials.get(user) == password Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

SCREENSHOT OF GENERATED CODE:



```
#Task2
def login(user, password, credentials):
    return credentials.get(user) == password

#Docstring Documentation with complete code
def login(user, password, credentials):
    """
    This function checks if the provided username and password match the credentials stored in a dictionary.

    Parameters:
    user (str): The username to be checked.
    password (str): The password to be checked.
    credentials (dict): A dictionary containing username-password pairs.

    Returns:
    bool: True if the username and password match the credentials, False otherwise.
    """
    return credentials.get(user) == password
#Example usage
credentials = {'user1': 'pass1', 'user2': 'pass2'}
print(login('user1', 'pass1', credentials))  # Output: True
print(login('user1', 'wrongpass', credentials))  # Output: False

#Inline Documentation with complete code
def login(user, password, credentials):
    """
    This function checks if the provided username and password match the credentials stored in a dictionary.

    Parameters:
    user (str): The username to be checked.
    password (str): The password to be checked.
    credentials (dict): A dictionary containing username-password pairs.

    Returns:
    bool: True if the username and password match the credentials, False otherwise.
    """
    # Use the get method to retrieve the password for the given user and compare it with the provided password
    return credentials.get(user) == password
#Example usage
credentials = {'user1': 'pass1', 'user2': 'pass2'}
print(login('user1', 'pass1', credentials))  # Output: True
print(login('user1', 'wrongpass', credentials))  # Output: False

#Google-Style Documentation with complete code
def login(user, password, credentials):
    """
    Checks if the provided username and password match the credentials stored in a dictionary.

    Args:
        user (str): The username to be checked.
        password (str): The password to be checked.
        credentials (dict): A dictionary containing username-password pairs. The keys are usernames and the values are passwords.
    Returns:
        bool: True if the username and password match the credentials, False otherwise.
    """
    return credentials.get(user) == password
#Example usage
credentials = {'user1': 'pass1', 'user2': 'pass2'}
print(login('user1', 'pass1', credentials))  # Output: True
print(login('user1', 'wrongpass', credentials))  # Output: False
```

```
(base) navedahmedshaik@Naveds-MacBook-Air AIAC % /usr/local/bin/python3 /Users/navedahmedshaik/Documents/3-2/AIAC/lab.9.1.py
The maximum value in the list is: 9
The maximum value in the list is: 9
True
False
True
False
True
False
(base) navedahmedshaik@Naveds-MacBook-Air AIAC %
```

EXPLANATION:

The function `login` takes three parameters: `user`, `password`, and `credentials`. It checks if the provided username and password match the credentials stored in a dictionary. The function uses the `get` method of the dictionary to retrieve the password associated with the given username and compares it with the provided password. If they match, it returns `True`; otherwise, it returns `False`. The function is documented using three different styles: docstring documentation, inline documentation, and Google-style documentation, providing clear explanations of the function's purpose, parameters, return value, and potential exceptions.

Style	Advantages	Disadvantages	Best Use Cases
Docstring (standard)	Simple, readable, built into Python help tools	Less structured for large projects	Small scripts, student projects
Inline comments	Explain logic directly near code	Not visible via help(), can clutter code	Complex internal logic explanation

Google-style	Structured, professional, IDEfriendly, great for teams	Slightly longer to write	Libraries, APIs, collaborative projects

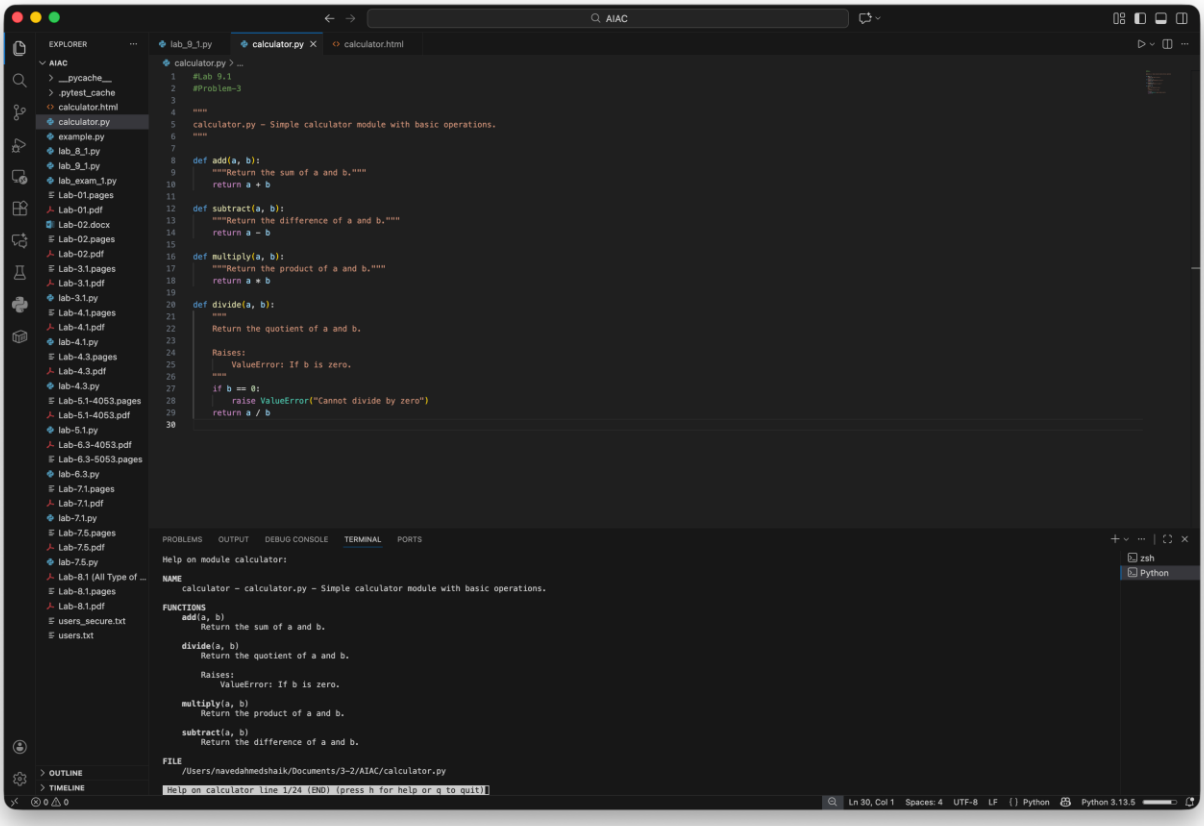
Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings: o add(a, b) – returns the sum of two numbers o subtract(a, b) – returns the difference of two numbers o multiply(a, b) – returns the product of two numbers o divide(a, b) – returns the quotient of two numbers
2. Display the module documentation in the terminal using Python's documentation tools.
3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

SCREENSHOT OF GENERATED CODE:



index
calculator /Users/navedahmedshaik/Documents/3-2/AIAC/calculator.py

calculator.py - Simple calculator module with basic operations.

Functions

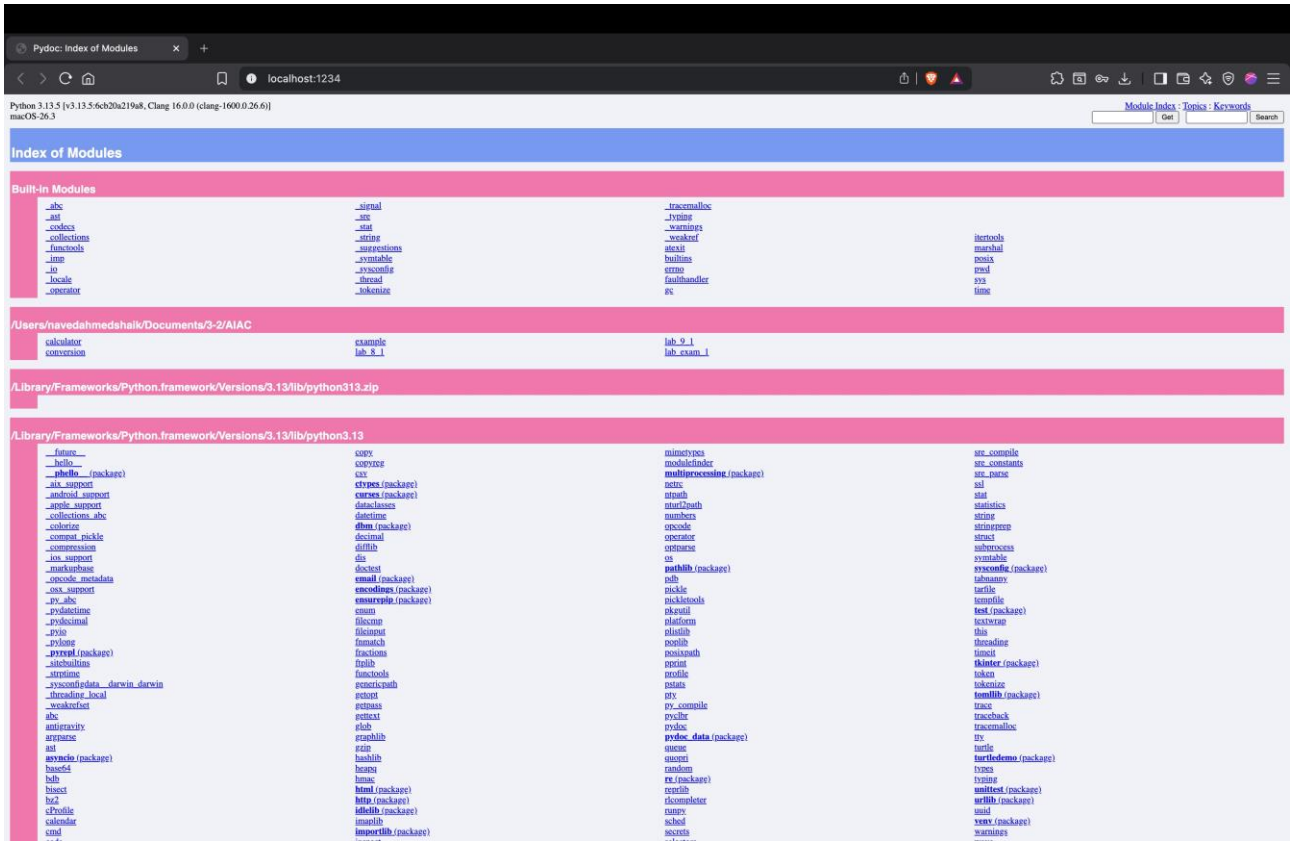
add(a, b)
Return the sum of a and b.

divide(a, b)
Return the quotient of a and b.

Raises:
ValueError: If b is zero.

multiply(a, b)
Return the product of a and b.

subtract(a, b)
Return the difference of a and b.

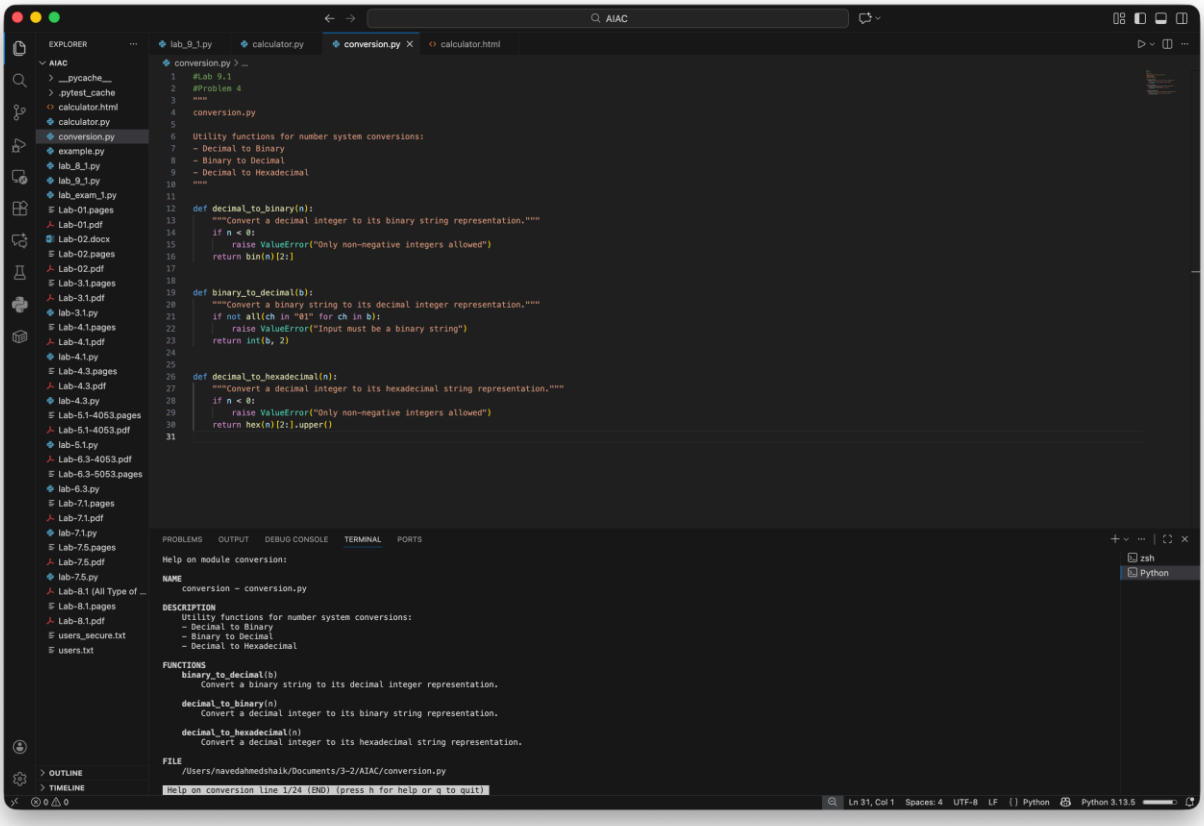


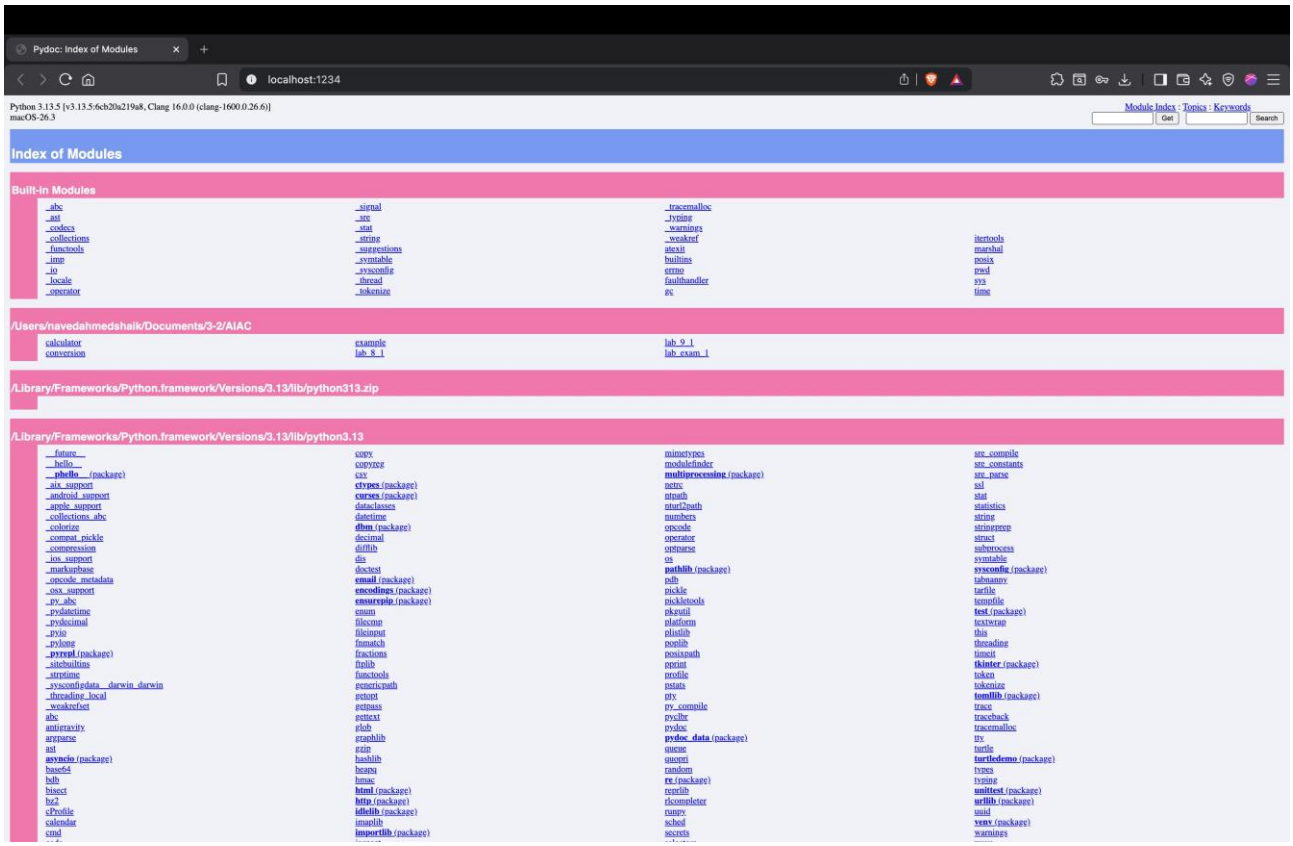
Problem 4: Conversion Utilities Module

Task:

1. Write a module named conversion.py with functions: o decimal_to_binary(n) o binary_to_decimal(b) o decimal_to_hexadecimal(n)
2. Use Copilot for auto-generating docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

SCREENSHOT OF GENERATED CODE:


AIAC
EXPLORER
lab_9_1.py
calculator.py
conversion.py
calculator.html
conversion.py > ...
#Lab 9.1
#Problem 4
"""
conversion.py
Utility functions for number system conversions:
- Decimal to Binary
- Binary to Decimal
- Decimal to Hexadecimal
"""
def decimal_to_binary(n):
"""Convert a decimal integer to its binary string representation."""
if n < 0:
raise ValueError("Only non-negative integers allowed")
return bin(n)[2:]
def binary_to_decimal(b):
"""Convert a binary string to its decimal integer representation."""
if not all(ch in "01" for ch in b):
raise ValueError("Input must be a binary string")
return int(b, 2)
def decimal_to_hexadecimal(n):
"""Convert a decimal integer to its hexadecimal string representation."""
if n < 0:
raise ValueError("Only non-negative integers allowed")
return hex(n)[2:].upper()
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Help on module conversion:
NAME
conversion - conversion.py
DESCRIPTION
Utility functions for number system conversions:
- Decimal to Binary
- Binary to Decimal
- Decimal to Hexadecimal
FUNCTIONS
binary_to_decimal(b)
Convert a binary string to its decimal integer representation.
decimal_to_binary(n)
Convert a decimal integer to its binary string representation.
decimal_to_hexadecimal(n)
Convert a decimal integer to its hexadecimal string representation.
FILE
/Users/navedahmedshaik/Documents/3-2/AIAC/conversion.py
Help on conversion line 1/24 (END) (press h for help or q to quit)
Ln 31, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.13.5


Pydoc: Index of Modules
localhost:1234
Python 3.13.5 [v3.13.5:6cb20a219a8, Clang 16.0.0 (clang-1600.0.26.6)]
macOS-26.3
Module Index : Topics : Keywords
Index of Modules
Built-in Modules
/Users/navedahmedshaik/Documents/3-2/AIAC
calculator
conversion
example
lab_8_1
lab_9_1
lab_exam_1
/Library/Frameworks/Python.framework/Versions/3.13/lib/python313.zip
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13

index
conversion /Users/navedahmedshaik/Documents/3-2/AIAC/conversion.py

conversion.py

Utility functions for number system conversions:
- Decimal to Binary
- Binary to Decimal
- Decimal to Hexadecimal

Functions

binary_to_decimal(b)
Convert a binary string to its decimal integer representation.

decimal_to_binary(n)
Convert a decimal integer to its binary string representation.

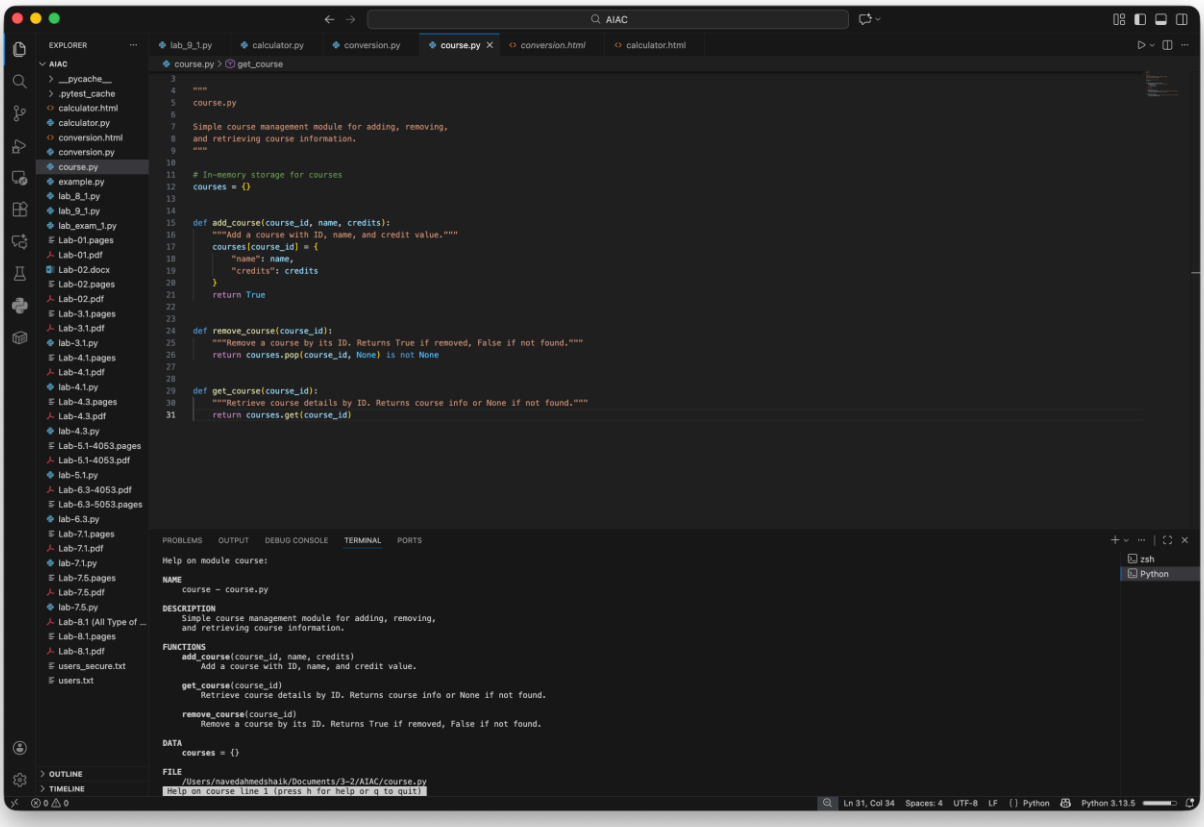
decimal_to_hexadecimal(n)
Convert a decimal integer to its hexadecimal string representation.

Problem 5 – Course Management Module

Task:

1. Create a module course.py with functions: o add_course(course_id, name, credits) o remove_course(course_id) o get_course(course_id)
2. Add docstrings with Copilot.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

SCREENSHOT OF GENERATED CODE:



index
course /Users/navedahmedshaik/Documents/3-2/AIAC/course.py

course.py

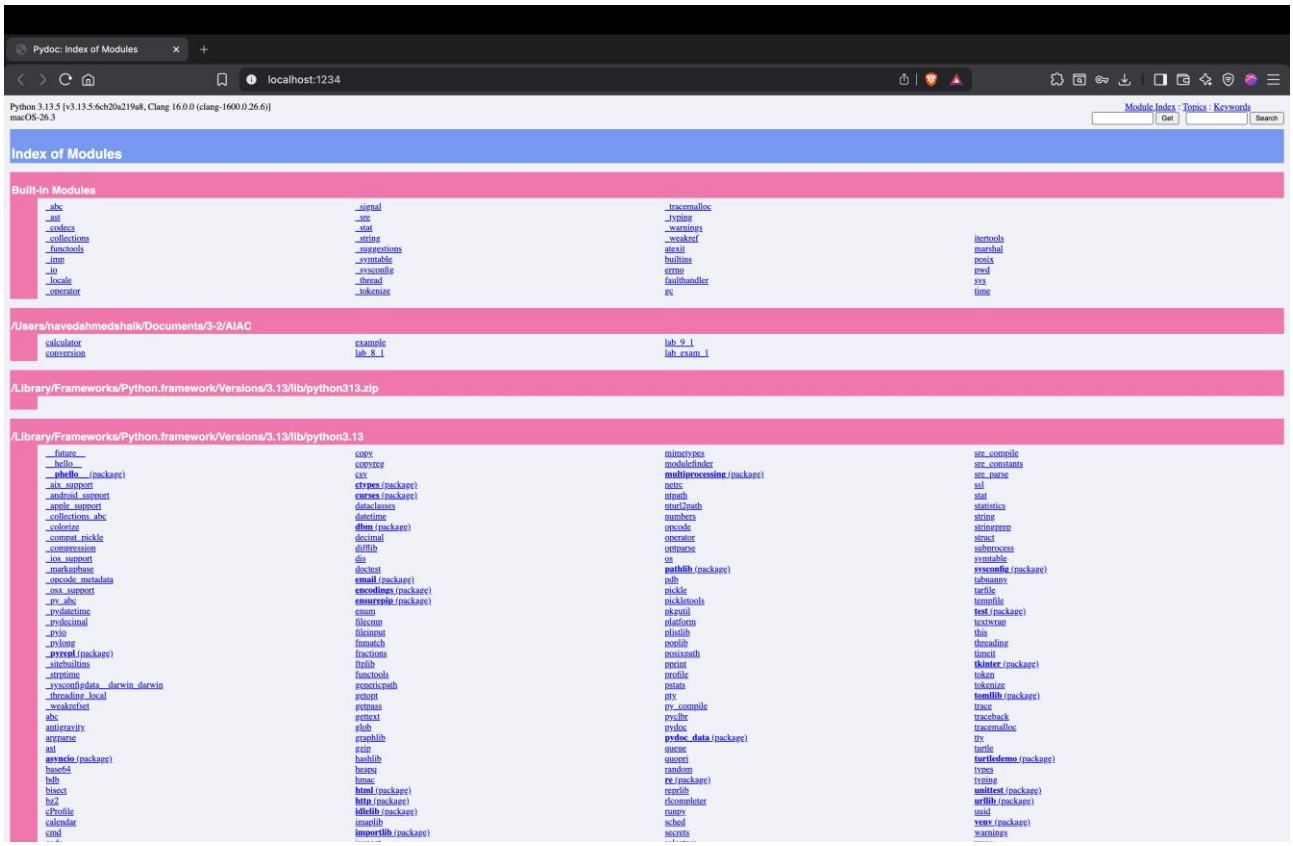
Simple course management module for adding, removing,
and retrieving course information.

Functions

add_course(course_id, name, credits)
Add a course with ID, name, and credit value.

get_course(course_id)
Retrieve course details by ID. Returns course info or None if not found.

remove_course(course_id)
Remove a course by its ID. Returns True if removed, False if not found.

Data

courses = {}